Practical Session 1: From Text to Vectors

Hands-On: TF-IDF, Cosine Similarity, and Word Embeddings

Juan F. Imbet

Paris Dauphine – PSL University

June 20, 2026

Session Overview

Duration: 1 hour **Format:** guided problems + Python case study

1. **Recap** (5 min) — key formulas from the lecture
2. **Problem 1** (15 min) — TF-IDF by hand on earnings call snippets
3. **Problem 2** (10 min) — interpreting word embedding analogies
4. **Case Study** (20 min) — Python: TF-IDF and cosine similarity on real financial text
5. **Discussion** (5 min) — when BoW vs. embeddings?
6. **Solutions** (5 min) — walkthrough

Notebook: `code/notebooks/01-intro/practical.ipynb`

Recap: TF-IDF Formulas

$$\mathrm{tf}(v,d) = \frac{\mathrm{count}(v,d)}{|d|} \qquad \mathrm{idf}(v,\mathcal{C}) = \log\left(\frac{|\mathcal{C}|}{\mathrm{df}(v,\mathcal{C})}\right)$$

$$\mathrm{tfidf}(v,d,\mathcal{C}) = \mathrm{tf}(v,d) \times \mathrm{idf}(v,\mathcal{C})$$

Document similarity (cosine, length-invariant):

$$\mathrm{sim}(d_i,d_j) = \frac{\tilde{\mathbf{x}}(d_i)^\top \tilde{\mathbf{x}}(d_j)}{\|\tilde{\mathbf{x}}(d_i)\|_2 \, \|\tilde{\mathbf{x}}(d_j)\|_2}$$

Key intuitions:

- Word in *all* documents $\Rightarrow$ IDF $= 0$ (suppressed)
- Word in *one* document $\Rightarrow$ IDF $= \ln N$ (maximally discriminative)
- TF-IDF picks out words that are *frequent in a document but rare in the corpus*

Recap: Word2Vec Skip-Gram with Negative Sampling

Skip-Gram objective (predict context from centre):

$$J_{\text{SG}} = \frac{1}{T} \sum_{t} \sum_{k \neq 0}^{\pm c} \log P(w_{t+k} \mid w_t)$$

Negative sampling approximation:

$$\mathcal{L}_{\text{NEG}}(w_t, w_{t+k}) = \log \sigma(\mathbf{v}_{w_{t+k}}^{\top} \hat{\mathbf{v}}_{w_t}) + \sum_{j=1}^{K} \mathbb{E}_{w_j \sim P_n} \left[\log \sigma(-\mathbf{v}_{w_j}^{\top} \hat{\mathbf{v}}_{w_t}) \right]$$

Key properties:

- Only $K+1$ embedding rows updated per step (vs. V for full softmax)
- Positive context vectors pushed *toward* centre; negative samples pushed *away*
- Linear analogies emerge: $\hat{\mathbf{v}}_{\text{king}} - \hat{\mathbf{v}}_{\text{man}} + \hat{\mathbf{v}}_{\text{woman}} \approx \hat{\mathbf{v}}_{\text{queen}}$

Problem 1: Setup

Consider the following four earnings call snippets. After lower-casing and removing stop words {the, a, of, in, by, is, our, we, to, and}:

d	Pre-processed tokens
d_1	`revenue increased strong margin growth`
d_2	`margin pressure volume declined uncertainty`
d_3	`strong volume revenue outlook positive`
d_4	`guidance withdrawn uncertainty pressure growth`

Vocabulary ($V = 10$): `declined, growth, guidance, increased, margin, outlook, positive, pressure, revenue, strong, uncertainty, volume, withdrawn` (remove duplicates; $V = 13$)

Problem 1: Tasks

Working individually or in pairs:

1. **Compute** $\mathrm{tf}(v, d)$ for each token v and document d. *Hint:* each document has 5 tokens, so $\mathrm{tf} \in \{0, 0.2\}$.
2. **Compute** $\mathrm{df}(v, \mathcal{C})$ and $\mathrm{idf}(v, \mathcal{C}) = \ln(4/\mathrm{df}(v))$ for each vocabulary word.
3. **Compute** $\mathrm{tfidf}(v, d_1, \mathcal{C})$ and $\mathrm{tfidf}(v, d_2, \mathcal{C})$ for all v.
4. **Compute** $\mathrm{sim}(d_1, d_2)$ using ℓ_2-normalised TF-IDF vectors. *Hint:* only non-zero entries contribute to the dot product.
5. **Interpret:** which two documents are most similar? Does this match your intuition about their content?

Time: 15 minutes.

Problem 2: Setup and Tasks

Suppose you have a Word2Vec model trained on a large corpus of earnings call transcripts. The model has learned the following (approximate) relationships in embedding space:

Analogy	Relationship
$\texttt{bond} - \texttt{coupon} + \texttt{dividend} \approx ?$	?
$\texttt{long} - \texttt{profit} + \texttt{loss} \approx ?$	?
$\texttt{inflation} - \texttt{high} + \texttt{low} \approx ?$	?

Tasks:

1. For each row, predict the missing word and explain the financial reasoning.
2. Explain *why* these linear relationships emerge from the Skip-Gram training objective.
3. Give an example of an analogy that would *fail* or produce a nonsensical result, and explain why.

Time: 10 minutes. Discuss with a neighbour.

Case Study: TF-IDF on Earnings Call Snippets

Goal: Use `scikit-learn` to compute TF-IDF representations of five real earnings call snippets and identify the most similar pair. **Setup:**

- Five snippets from Q3 2023 earnings calls (provided in the notebook)
- Apply `TfidfVectorizer` with `stop_words='english', max_features=500`
- Compute pairwise cosine similarity matrix
- Identify and interpret the most similar and least similar pairs

Open `code/notebooks/01-intro/practical.ipynb` and run cells 1–4.

Case Study: Python Code Skeleton

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# snippets: list of 5 earnings call excerpt strings
vectorizer = TfidfVectorizer(stop_words='english',
                             max_features=500)
X = vectorizer.fit_transform(snippets)  # (5, 500) sparse matrix

# Pairwise cosine similarity
sim_matrix = cosine_similarity(X)
np.fill_diagonal(sim_matrix, 0)  # zero out self-similarity

# Most similar pair
i, j = np.unravel_index(sim_matrix.argmax(), sim_matrix.shape)
print(f"Most similar: doc {i} and doc {j}, sim = {sim_matrix[i,j]:.3f}")

# Top terms for doc i
feature_names = vectorizer.get_feature_names_out()
top_terms = feature_names[X[i].toarray().argsort()[0, -5:]]
print(f"Top terms for doc {i}: {top_terms}")
```

Case Study: Questions to Answer

After running the notebook:

1. Which pair of documents has the highest cosine similarity? What do their top TF-IDF terms have in common?

2. The similarity between two documents from *different* companies turns out to be higher than the similarity between two documents from the *same* company but in different sectors. How can you explain this?

3. Change `max_features` from 500 to 50. Does the ranking of most-similar pairs change? What does this tell you about the sensitivity of TF-IDF to vocabulary truncation?

A stretch extension comparing TF-IDF with averaged GloVe embeddings appears in the appendix.

Discussion: When BoW vs. Embeddings?

The central trade-off

TF-IDF is interpretable, fast, and requires no pre-training.
Word embeddings are dense, semantically rich, but are black-box vectors.

Discuss in groups (5 min):

1. You are building a system to flag 10-K filings that are unusually negative. Would you use TF-IDF + LM dictionary or word embeddings? Justify.
2. You are trying to retrieve, from a database of 500,000 earnings call transcripts, all calls that discuss supply-chain risk. Which method would you prefer?
3. An auditor wants a system that can *explain* why a particular filing was flagged. Which method is easier to explain?
4. Under what conditions might TF-IDF actually *outperform* embeddings in a return prediction task?

Solution: Problem 1 (TF-IDF)

Document frequencies and IDF ($N = 4$, ln base):

Token	df	idf	tfidf in d_1 (tf = 0.2)
declined	1	$\ln 4 \approx 1.386$	0
growth	2	$\ln 2 \approx 0.693$	$0.2 \times 0.693 \approx 0.139$
guidance	1	1.386	0
increased	1	1.386	$0.2 \times 1.386 \approx 0.277$
margin	2	0.693	$0.2 \times 0.693 \approx 0.139$
pressure	2	0.693	0
revenue	2	0.693	$0.2 \times 0.693 \approx 0.139$
strong	2	0.693	$0.2 \times 0.693 \approx 0.139$
uncertainty	2	0.693	0
volume	2	0.693	0

$\text{sim}(d_1, d_2) \approx 0.21$ (shared: `margin`). Most similar pair: d_2 and d_4 (shared: `uncertainty, pressure, growth`).

Solution: Problem 2 (Analogies)

Analogy solutions:

- $\texttt{bond} - \texttt{coupon} + \texttt{dividend} \approx \texttt{equity}$
  Bond pays coupons; equity pays dividends. Subtracting the income-type distinguishes the instrument.
- $\texttt{long} - \texttt{profit} + \texttt{loss} \approx \texttt{short}$
  Long profits from price increases; short profits from price decreases, i.e. losses for a long.
- $\texttt{inflation} - \texttt{high} + \texttt{low} \approx \texttt{deflation}$
  Directional dimension in the macro vocabulary.

Why these work: co-occurrence patterns encode systematic financial relationships. Words that appear in the same contexts as "bond" but not in the contexts of "coupon" tend to be equity-related terms.

Session Wrap-Up

Session Summary

TF-IDF: fast, interpretable, order-invariant.
Word2Vec/GloVe: semantic geometry, linear analogies, domain-specific adaptation needed.
Next lecture: Transformers learn *context-dependent* embeddings — no static vectors.

Appendix

Stretch Problem: TF-IDF vs. GloVe Embeddings

Extra / optional material — beyond the core lecture.

Stretch: TF-IDF vs. Averaged GloVe Embeddings

Extension (optional): Load `glove-wiki-gigaword-100` via `gensim`. For each document, compute the average word embedding.

1. Does the embedding-based similarity ranking agree with the TF-IDF ranking?
2. What does *disagreement* between the two rankings tell you about what each representation captures?
3. When would you trust the embedding ranking over the TF-IDF ranking, and vice versa?